

```

# MAIN PIPELINE (only runs if checks pass)

# Imports
import os

from langchain_community.document_loaders import (
    TextLoader,
    PyPDFLoader,
    UnstructuredMarkdownLoader
)

from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_chroma import Chroma

def run_rag_pipeline(file_path):

    # 1. Load documents
    print(f"\n[1/4] Loading document: {file_path}")

    if file_path.endswith(".pdf"):
        loader = PyPDFLoader(file_path)

    elif file_path.endswith(".txt"):
        loader = TextLoader(file_path)

    elif file_path.endswith(".md"):
        loader = UnstructuredMarkdownLoader(file_path)

    else:
        raise ValueError(f"Unsupported file type: {file_path}")

    docs = loader.load()

    print(f"✅ Loaded {len(docs)} document(s)")

    # 2. Chunk documents
    print("\n[2/4] Chunking documents")

```

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    length_function=len,
    separators=["\n\n", "\n", " ", ""]
)

chunks = text_splitter.split_documents(docs)

print(f"✅ Loaded {len(chunks)} chunks")

# 3. Initialise embeddings
print("\n[3/4] Initialising embedding model")

embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2"
)

print("✅ Embedding model ready")

# 4. Create vector store
print("\n[4/4] Creating vector store")

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db"
)

print(f"✅ Vector store created with {len(chunks)} chunks")

# 5. Test a query
print("\n[TEST] Testing similarity search")

query = "What is this document about?"
```

```

results = vectorstore.similarity_search(query)

print(f"✅ Query: '{query}'")
print(f"✅ Found {len(results)} relevant chunks")

for i, result in enumerate(results, 1):
    print(f"\nResult {i}:")
    print(f"    {result.page_content[:150]}...")

print("\n" + "=" * 60)
print("✅ PIPELINE COMPLETE")
print("=" * 60)

print("\n✅ Vector store ready for queries!")
print("To query:")

return vectorstore

# MAIN EXECUTION
if __name__ == "__main__":

    file_path = "test_doc.txt"

    vectorstore = run_rag_pipeline(file_path)

```